Running Underworld in a Browser: Any Repository, Any Version

Louis Moresi[1]

[1]Australian National University

DOI https://doi.org/10.6084/m9.figshare.33216996

Keywords Underworld Code, Python/Jupyter

PUBLISHED Aug 11, 2026 ARTICLE UWTN 2026-010 VERSION 1.1.0 LICENCE CC-BY-4.0 ARCHIVED 2026-08-31T17:36:06Z SOURCE https://www.underworldcode.org/running-underworld-in-a-browser/

Somebody reads a paper and wants to run the model; they have forty minutes. They will not have time to install PETSc. They may not have a compiler. If the answer is "clone this, then build these dependencies", the answer is really: "no thanks".

Our solution to this is *one link*. It opens JupyterLab in a browser, with **any released version** of Underworld already running, and **any public repository** pulled in beside it. Those three choices are independent, and the repository being launched needs nothing added to it — no Dockerfile, no `.binder/` directory, no configuration at all.

This article describes the four pieces that make that work, and the one additional requirement peculiar to Underworld: it compiles C while a model runs, so the image has to carry a compiler.

1. The classroom problem

Forty minutes is a busy researcher's attention span and about the time it takes to go for coffee and forget what you were focused on. It's the upper limit. The case that drove this work was teaching a class, where the arithmetic is harsher.

I have watched a two-hour practical with thirty students go like this: forty minutes installing, forty minutes on the six laptops where the install went wrong, and the remainder on the actual tasks. Departmental lab machines fix this until the practical needs a version they do not have, or a student wants to continue at home.

What a class *actually needs* turns out to be modest:

- **Nothing installed.** A browser, on whatever the student owns.
- **Everyone on the same version**, all semester. If the practicals were written against `v3.1.0`, then `v3.1.0` is what they run in week nine, no matter what happened on `development` in the meantime.
- **One link per practical**, each opening the folder for that week, so nobody is navigating a file tree to find where they are supposed to be.

- **Corrections that take effect immediately.** Fix the notebook, push, and the next student to reload the page gets the fixes — no reissued handout.

Before university level, this is felt more acutely. A high school cannot repurpose a departmental research cluster, and teachers often have no permission to install *anything* on a managed device. A link opens the same way a video does. Some of what Underworld produces is useful well before undergraduate level — a fault slipping and the ground deforming around it, a slab sinking, plates pulling apart — a class that could never be asked to install a finite element code can be asked to click something and change a number to see what happens.

We ran our own cloud for exactly this problem. It worked, but it also needed somebody to run it, pay for it, and be available when it broke (a few minutes before a class).

There is a simpler way to do things !

2. The four pieces

Each does a single job:

1. A **container image** with Underworld already built, published to the GitHub Container Registry.
2. A **launcher repository** — almost empty, just instructions for firing up the containers on binder.
3. Two **GitHub workflows** that build the Underworld image for each release and modify the launcher repository to announce the new release.
4. **nbgitpuller**, which clones the reader's repository into a running session.

3. The container image

The image is built in stages and then stripped, because binder loves lightweight images and we need reliable, fast launches. Once the code is built, anything the run time does not need comes out of the container:

Removed	Saved
`docs_legacy`	229 MB
pixi package cache	∼500 MB
`conda-meta` metadata	24 MB
man pages, `__pycache__`, `*.pyc`, test suites	tens of MB

The git clone is `--depth 1 --single-branch`, which keeps `.git` at about 5 MB. It is kept rather than deleted, because a shallow history is still enough to `git pull` if we need updates for any reason.

The Linux/Underworld runtime is around 2.7 GB, but we have to split that into layers inside the container — another binder reliability measure. What we can't delete is the compiler toolchain and the C header files because Underworld compiles sympy to C code when it runs.

4. The launcher repository

`underworldcode/uw3-binder-launcher` contains, per branch, a `.binder/Dockerfile` of two meaningful lines:

```
FROM ghcr.io/underworldcode/uw3-base:v3.1.0-slim
ENV UW3_BRANCH=v3.1.0
```

That is the whole thing. It exists, rather than binder being pointed straight at the Underworld repository, because mybinder caches on the commit hash of the repository it launches. Underworld changes daily, and so does any repository where you are working on content (like the class you are teaching !)

The launcher almost never changes, so the cache almost always hits, and the Underworld example arrives as a pre-built image. A first launch after a release is slow; launches after that are quick.

5. The release workflows

This is the fiddly part. In the Underworld repository, `binder-image.yml` runs whenever something in the repository requires the container to be rebuilt e.g. a new release tag is created or a push is made to the main branch. It then builds the image, pushes it to the GitHub Container Registry tagged for the branch or release, and notifies the launcher:

```
- name: Trigger launcher update
  uses: peter-evans/repository-dispatch@v2
  with:
    repository: underworldcode/uw3-binder-launcher
    event-type: image-updated
    client-payload: '{"branch": "...", "ref_type": "..."}'
```

In the ***launcher*** repository, `update-image.yml` listens for that and behaves differently according to what arrived:

- **A branch push** updates the existing launcher branch's `Dockerfile` to point at the new image. `main` and `development` therefore track the Underworld code repository.
- **A release tag** creates a *new launcher branch* named for the tag, containing a frozen `Dockerfile` pinned to that release's image.

A release branch is written once, and nothing afterwards changes it. `v0.99` will still be `v0.99` in five years. The release and its launcher are made in the same run, so they cannot drift apart.

6. nbgitpuller

The launcher image carries nbgitpuller, which clones a repository into the session at start-up and merges updates on later launches. That has three consequences:

- A repository needs **no** binder configuration. The environment comes from the launcher; the notebooks come from the third-party repository.
- It is pulled **fresh on every launch**, so a correction pushed now is live for the next person who clicks the launch link.
- The repository requirements are: public on GitHub, notebooks using the `python3` kernel, and `import underworld3 as uw`.

7. The URL

The link says three things: which Underworld version, which repository, and where to start inside that repository.

```
https://mybinder.org/v2/gh/underworldcode/uw3-binder-launcher/VERSION
    ?urlpath=git-pull
     &repo=https://github.com/USER/REPO
     &branch=BRANCH
     &urlpath=lab/tree/REPO/WHERE
```

Part	What it selects
`VERSION`	the launcher branch: `main`, `development`, or a release such as `v3.1.0`
`repo`	the repository to clone alongside Underworld
`branch`	which branch of it
the second `urlpath`	where JupyterLab opens: a folder, or one notebook

Both `urlpath` parameters are needed. The first tells binder to hand over to nbgitpuller; the second is nbgitpuller's own instruction about where to land once it has finished cloning.

The escaping. That plain form is not what gets pasted. It is a URL nested inside a URL, so everything after `git-pull` must be percent-encoded — and the repository address, one level deeper again, is encoded twice. `/` becomes `%2F` at one level and `%252F` at two. That is why the working links look the way they do, and why we wrote a script for it (that forty-minute rule applies here too) !

```
python scripts/binder_wizard.py myuser/my-course main tutorials/intro.ipynb
```

That emits the encoded URL and a ready-to-paste badge in Markdown, HTML or reStructuredText — which is how a **Launch** button gets onto a course or paper repository.

8. Setting up a course

Put the practicals in one public repository, a folder per week:

```
geodynamics-2026/
    week-01-convection/
    week-02-rheology/
    week-03-subduction/
```

Then issue one link per week, identical apart from the folder, and all naming the same release:

```
.../uw3-binder-launcher/v3.1.0?...&urlpath=lab/tree/geodynamics-2026/week-01-convection
.../uw3-binder-launcher/v3.1.0?...&urlpath=lab/tree/geodynamics-2026/week-02-rheology
```

Nothing else is needed: no accounts, no lab image, no install instructions, and no version drift over the semester. A fix pushed on Tuesday is what the Wednesday group gets.

One practical caution. mybinder.org is free and shared, and thirty simultaneous launches is a real load on it. The cache works in your favour (the first launch pulls the image and the rest are quick) so it is worth clicking the link yourself an hour before the class.

If mybinder.org is busy or down, the practical is down. So be careful relying on it for an exam !

There is no persistency between sessions, so make sure students save their work regularly to a drive they own.

9. Limits

The **environment** is guaranteed. Pinning to `v3.1.0` fixes Underworld, its dependencies, and the compiler that builds its generated C.

Data is a different matter. A notebook that downloads a dataset at run time is only as reproducible as that download. If it is critical data, put a sample version in the repository.

Three limits come with not running servers:

- **Sessions are ephemeral.** There is no home directory. Push the work to git or download it before closing the tab.
- **mybinder.org is a free, shared service.** It is busy sometimes, and it has memory and CPU limits. It is for teaching, demonstrating and trying things — not for production runs.
- **Public repositories only**, because nbgitpuller has to be able to see the notebooks to bring them over.